

Aufgabenblatt 7

Übung zu Einführung in die Bildverarbeitung

Hanna Beitz, Malin Haas, Lasse Huber-Saffer, Tim Radke,
Teodora Rogozenco, Jannis Scheff, Christian Wilms
SoSe 2026

Ausgabe: 5. Juni 2026 - **Abgabe bis: 12. Juni 2026, 10:00 per Moodle!**

Lernziele

- Faltung und Korrelation anwenden und unterscheiden können
- Einfluss des Paddings verstehen
- Auswirkung von Filtern zur Rauschreduktionen verstehen

0	0	0
0	1	0
0	0	0

Abbildung 1: Graustufenbild **I** für Aufgabe 1 mit Pixelwerten als Zahlen.

Aufgabe 1 — Faltung und Korrelation anwenden - $10+5+5+5 = 25$ Punkte - Theorieaufgabe

1. Gegeben sei das Bild **I** in Abbildung 1 und der Filterkern

$$\mathbf{K} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}. \quad (1)$$

Berechnet die Faltung $\mathbf{K} \star \mathbf{I}$ händisch. Normiert dazu den Kernel und wendet ausreichend zero-padding an.

2. Berechnet die Korrelation $\mathbf{K} \star \mathbf{I}$ händisch. Normiert dazu den Kernel und wendet ausreichend zero-padding an.
3. Welche Auswirkungen hat die Faltung eines beliebigen Kernels mit dem Bild **I**?
4. Meist werden Kernels zur Bildglättung so normiert, dass die Summe über sie 1 ist. Welchen Hintergrund hat das? Welche Konsequenzen hat bspw. die Anwendung eines Box-Filters, dessen Gewichte zusammen 0.5 oder 2 ergeben, auf ein Bild? Beantwortet diese Fragen kurz schriftlich und gebt Beispiele (auch in 1D möglich), die eure Thesen untermauern.

Aufgabe 2 — Eigenschaften von Faltung und Korrelation - $10+5+10 = 25$ Punkte - Theorieaufgabe

1. Wie in der Vorlesung vorgestellt, ist die Faltung kommutativ. Beweist dies!
2. Im Gegensatz zur Faltung ist die Korrelation i.A. nicht kommutativ. Zeigt dazu ein Gegenbeispiel, gerne auch in 1D.
3. Zeigt an einem Gegenbeispiel, dass der Median-Filter i.A. kein linearer Filter ist.

Aufgabe 3 — Rauschreduktion - 3+7+5+5+5 = 25 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.filters`, `skimage.util`, `matplotlib`

Die Faltung kann genutzt werden, um das Rauschen in einem Bild zu mindern. Dazu sollen verschiedene Filterkerne in Kombination mit verschiedenen Typen von Rauschen im Rahmen dieser Aufgabe erprobt werden.

1. Ladet zunächst das Bild `einstein.png` aus dem Moodle und verrauscht es mit Gaußischem Rauschen. Wandelt dafür zunächst den Wertebereich des Bildes von $0 \dots 255$ auf $0 \dots 1$ um. Nutzt zum Verrauschen die Funktion `skimage.util.random_noise()` mit dem Parameter `mode='gaussian'` und einer Varianz von 0.01. Zeigt das verrauschte Bild an.
2. Schreibt eine Funktion, die den mittleren absoluten Unterschied zwischen den korrespondierenden Pixeln zweier Bilder berechnet. Wie groß ist die mittlere absolute Differenz zwischen dem Originalbild (`einstein.png`) und eurer verrauschten Variante?
3. Wendet nun den Gauß-Filter mit unterschiedlichen Standardabweichungen und dem Padding *nearest* auf dasselbe verrauschte Bild an. Testet Standardabweichungen im Bereich $0.1, \dots, 2$ in Schritten von 0.1 und vergleicht das Ergebnis erneut mit Hilfe der mittleren Differenz mit dem Originalbild. Welcher Filter liefert das bessere Ergebnis (geringere mittlere Differenz zum Originalbild)? Visualisiert zudem das beste Ergebnis mit dem Gauß-Filter.
4. Verrauscht nun das Originalbild `einstein.png` mit Salt and Pepper Noise. Nutzt dazu erneut die Funktion `skimage.util.random_noise()` mit dem Parameter `mode='s&p'`. Die Wahrscheinlichkeit für das Auftreten von Salt and Pepper soll bei 0.1 liegen. Wendet nun den Gauß-Filter mit verschiedenen Standardabweichungen (erneut im Bereich $0.1, \dots, 2$ in Schritten von 0.1) auf das mit Salt and Pepper Noise verrauschte Bild an. Wie groß ist die geringste mittlere Distanz zum Originalbild? Visualisiert auch das Ergebnis.
5. Welcher Filter kann das mit Salt and Pepper Noise verrauschte Bild besser entrauschen? Wendet den Filter auf das Bild an. Wie verändert sich die mittlere Differenz des Ergebnisses zum Originalbild?

9	7	8	6	5	7	4	2	0	1
---	---	---	---	---	---	---	---	---	---

Abbildung 2: Bildzeile für Aufgabe 4 mit Pixelwerten als Zahlen.

Aufgabe 4 — Padding am Bilrand - 3+8+3+3+4+4 = 25 Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `matplotlib`

In der Vorlesung wurden verschiedene Möglichkeiten der Behandlung des Bildrands (Padding) im Rahmen der Faltung vorgestellt. Diese sollen nun erprobt und ihre Effekte an einem kleinen Beispiel untersucht werden. Dazu wird im Rahmen dieser Aufgabe nur eine einzige Bildzeile (s. Abbildung 2) betrachtet. Diskutiert in jeder Teilaufgabe die Auswirkungen des jeweiligen Paddings kurz schriftlich.

Hinweis 1: In dieser Aufgabe ist die Nutzung der Funktion `numpy.pad()` nicht erlaubt.

Hinweis 2: Schreibt eure Padding-Funktionen so, dass sie für beliebige Bildzeilen wie die obige funktionieren.

1. Setzt die Bildzeile aus Abbildung 2 in Python um. Plottet die Bildzeile anschließend als Funktion mit `matplotlib.pyplot.plot`.
2. Definiert einen 1D-Filterkern

$$k = [1 \quad 1 \quad 1 \quad 1 \quad 1]$$

in Python und normiert ihn, sodass die Summe der Einträge 1 ist. Implementiert nun selbst die 1D-Faltung zwischen der Bildzeile und dem Filterkern k . Wendet dafür zunächst zero-padding an. Die Größe des Paddings (Anzahl der ergänzten Werte) soll dabei nicht explizit angegeben werden, sondern aus dem Filterkern ermittelt werden. Fügt das Ergebnis ebenfalls in den oben erstellten Plot ein. Welche Auswirkung hat das zero-padding auf die Randbereiche?

3. Ändert das Padding nun so, dass ein constant padding mit der Konstante 9 genutzt wird. Faltet erneut die Bildzeile mit k und plottet das Ergebnis. Wie verändert es sich?
4. Nutzt nun das replicate padding statt der konstanten paddings. Wie verändert sich das Ergebnis der Faltung von Bildzeile und k hier und im allgemeinen Fall? Plote zudem das Ergebnis.
5. Verändert das Padding nun zu reflect padding. Plottet auch das Ergebnis der Faltung zwischen der Bildzeile und k mit diesem Padding und beschreibt kurz die Veränderung.

6. Nutzt zuletzt die Variante *wrap around* paddings. Faltet die Bildzeile erneut mit k und plottet das Ergebnis. Diskutiert auch hier kurz die Auswirkungen des Paddings.

Zusatzaufgabe 5 — Saliency Detection mit Gauß-Filtern - $10+5 = 15$ Punkte - Programmieraufgabe

Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.filters`, `skimage.transform`, `matplotlib`

Betrachtet erneut die Aufgabe 4 von Aufgabenblatt 5 zur Saliency Detection.

1. Ändert eure Implementation so, dass statt der quadratischen Zentrums- und Umgebungsregionen Gauß-Filter genutzt werden, um diese nun runden Regionen abzubilden. Experimentiert mit den Parametern des Filters, sodass ihr ähnliche Ergebnisse wie in der Ursprungsaufgabe erzeugt (hohe Salienz für die Steine, die Punkte bzw. den gesamten Marienkäfer).
2. Welche Vorteile hat die Nutzung der Gauß-Filter? Nennt mindestens einen. Welcher Filter wurde in der bisherigen Lösung genutzt?